

university of groningen

Performance Engineering: Tutorial #02

WMCS045-05

Dirk Pleiter

Bernoulli Institute

2026-09-10

Overview



1. Project Overview

2. Applications

Content



1. Project Overview

2. Applications

Assignment Overview



- **What is expected?**
  - Design and implement a **performance analysis** process for a selected application
  - Interpret performance analysis results using **performance modelling** techniques
  - Identify, implement, and assess solutions for **performance improvement or optimization**
- **What is not expected?**
  - Produce the fastest possible code
- **Your choices**
  - Application
  - Target computing system(s)
  - Performance tools
  - Performance modelling techniques
  - Performance improvement and optimization techniques

Organization

- **Steps and timeline**
  1. Select application
  2. Prepare work plan (deadline: 18.09.2026)
  3. Execute work plan
  4. Finish draft report (deadline: 16.10.2026)
  5. Present results (20+22.10.2026)
  6. Submit final report (deadline: 02.11.2026)
- **Presentation**: 20 minutes presentation + 5 minutes Q&A
- **Report**:
  - 10-15 pages
  - At most 20 % for figures
  - Annex is possible but will not be assessed
  - Code to be provided separately (reference to git repository is an option)

Task Details (1/2)



- **Application selection**
  - Proposals provided in next section
  - Your are free to propose your own application, but cross-check with lecturer
- **Performance analysis**
  - Use at least 2 different tools for performance analysis
  - Foresee checks to verify correctness of the tools
  - Identify and describe all performance critical kernels
- **Performance modelling**
  - Justify the choice of the performance modelling methodology
  - Make a decent effort to determine the attainable performance using suitable benchmarks

Task Details (2/2)



- **Performance improvement and optimization**
  - Choose several performance improvement or optimization strategies
  - *Note*: Good justification of the strategy is more important than actual speed-up figures
- **Documentation**
  - Provide a copy of all code
    - Reference to a repository is sufficient
  - Document the computing systems and the development environment that have been used

Content



1. Project Overview

2. Applications

Proxy Applications



- **Proxy applications** = Simplified version of an application where for a given co-design purpose relevant features are preserved
  - They have become a popular co-design instrument in HPC
- Benefits of proxy applications (aka mini-apps): [M.A. Heroux; 2009]
  - Facilitate interaction with application's research community
  - Make application simulation feasible
  - Explore new languages and programming models
  - Support compiler optimization
- Critique and alternative approach: Octopodes [S. Matsuoka et al.;2022]
  - **Octopodes** = Highly parametrisable, easily amendable, motif-like representations of algorithms or kernels
  - Goal: Capture the algorithms or complex operations instead of the implementation

2-Dimensional Poisson



- **Description**: Solve the Poisson equation in 2 dimensions using the iterative Jacobi algorithm
  - Very simple stencil-type application
  - Widely used for training purposes
  - It is recommended to parallelise the application
- **Dwarf**: Structure grids
- **Source code**:
  `https://github.com/haicgu/training/tree/main/HPC/hands-on/poisson2d`
- **Literature**
  - `https://gpuopen.com/learn/amd-lab-notes/amd-lab-notes-jacobi-readme/`

miniFE



- **Description**: MiniFE is an proxy application for unstructured implicit finite element codes
  - Implementation of an iterative solver, i.e. sparse-vector multiplications dominate
  - A selection between different implementations needs to be made
- **Dwarf**: Sparse linear algebra
- **Source code**: `https://proxyapps.exascaleproject.org/app/minife/`
- **Literature**
  - P.T. Lin et al., "Assessing a mini-application as a performance proxy for a finite element method engineering application", 2015, `https://doi.org/10.1002/cpe.3587`
  - J. Luitjens et al., "Optimizing miniFE an Implicit Finite Element Application on GPUs", `https://developer.download.nvidia.com/GTC/PDF/GTC2012/PresentationPDF/S0302-GTC2012-Accelerating-miniFE-Element.pdf`

miniMD



- **Description**: miniMD is a parallel molecular dynamics (MD) simulation package written in C++
  - A selection between different implementations needs to be made
  - A newer version is available: ExaMiniMD
- **Dwarf**: N-body methods
- **Source code**: `https://proxyapps.exascaleproject.org/app/minimd/`
- **Literature**
  - P.S. Crozier, "LAMMPS/miniMD overview: communica5on/computa5on performance", 2010, `https://mantevo.github.io/pdfs/LAMMPS.pdf`

NEKbone



- **Description**: Nekbone captures the basic structure and user interface of Nek5000, a high order, incompressible Navier-Stokes solver based on the spectral element method
  - Nek5000 is a widely used computational fluid dynamics code
- **Dwarf**: Structured grids, dense linear algebra
- **Source code**: `https://proxyapps.exascaleproject.org/app/nekbone/`
- **Literature**
  - JaeHyuk Kwack et al., "Roofline-based Performance Efficiency of HPC Benchmarks and Applications on Current Generation of Processor Architectures", 2019, `https://cug.org/proceedings/cug2019_proceedings/includes/files/pap115s2-file1.pdf`

CloverLeaf



- **Description**: CloverLeaf is a Lagrangian-Eulerian hydrodynamics mini-app
  - Stencil-type application
  - A selection between different implementations needs to be made
- **Dwarf**: Structured grids
- **Source code**: `https://uk-mac.github.io/CloverLeaf/`
- **Literature**
  - J. Laukemann et al., "CloverLeaf on Intel Multi-Core CPUs: A Case Study in Write-Allocate Evasion", 2023, `https://doi.org/10.48550/arXiv.2311.04797`

GABPS



- **Description**: Benchmark suite for graph-algorithms like Breadth-First Search (BFS)
  - It is recommended to focus on the `bfs` benchmark
- **Dwarf**: Graph traversal
- **Source code**: `https://github.com/sbeamer/gapbs`
- **Literature**
  - S. Beamer et al., “The GAP Benchmark Suite”, 2015, `https://doi.org/10.48550/arXiv.1508.03619`

Questions?



[Wiki Media]